

1. Create employee Ali

The screenshot shows the Postman application interface. The top bar indicates the workspace is 'Ahed's Workspace'. The left sidebar shows a 'REQUEST HISTORY' with several requests. The main panel displays a POST request to 'http://localhost:8080/api/employees'. The 'Headers' section shows 'Content-Type' set to 'application/json'. The 'Body' section shows a JSON object:

```
{  "id": 1,  "name": "Ali",  "email": "ali@co.com"}
```

. The response status is '201 Created' with a response time of 98 ms and a body size of 211 B. The response body is shown in JSON format.

POST **Create employee Ali** Save Share

POST http://localhost:8080/api/employees Send

Headers 8 hidden Cookies

Key	Value	Bulk Edit	Presets
Content-Type	application/json		
Key	Value	Description	

Body JSON 201 Created 98 ms 211 B

```
{  "id": 1,  "name": "Ali",  "email": "ali@co.com"}
```

2. Create a resource

The screenshot shows the Postman application interface. The top bar indicates the workspace is 'Ahed's Workspace'. The left sidebar shows a 'REQUEST HISTORY' with several requests. The main panel displays a POST request to 'http://localhost:8080/api/resources'. The 'Body' section shows a JSON object:

```
{  "id": 1,  "title": "Clean Code",  "type": "BOOK",  "totalCopies": 1,  "availableCopies": 1}
```

. The response status is '201 Created' with a response time of 710 ms and a body size of 248 B. The response body is shown in JSON format.

POST **Create a resource** Save Share

POST http://localhost:8080/api/resources Send

Params Cookies

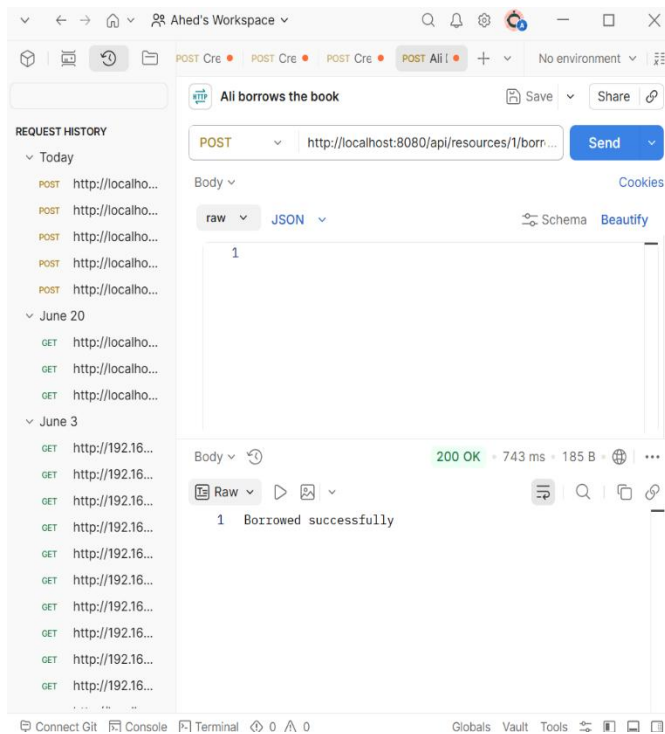
Query Params

Key	Value	Desc...	Bulk Edit
Key	Value	Description	

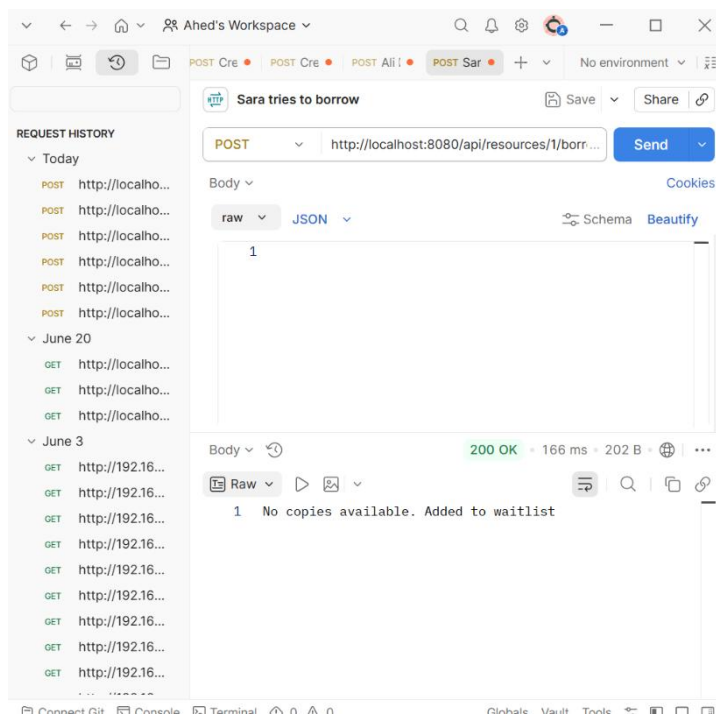
Body JSON 201 Created 710 ms 248 B

```
{  "id": 1,  "title": "Clean Code",  "type": "BOOK",  "totalCopies": 1,  "availableCopies": 1}
```

3. Ali borrows the book



4. Sara tries to borrow (no copies left, joins waitlist)



5. Ali returns the book (reservation created for sara)

The screenshot shows a REST client interface with a request titled "Ali returns the book". The request is a POST to `http://localhost:8080/api/resources/1/retui...`. The headers section shows "Content-Type" set to "application/json". The response is a 200 OK status with a body of "1 Returned successfully".

REQUEST HISTORY

- Today
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
- June 20
 - GET http://localho...
 - GET http://localho...
 - GET http://localho...
- June 3
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...

Headers

Key	Value	Bulk Edit	Presets
Content-Type	application/json		

Body

1 Returned successfully

6. Sara claims her reservation

The screenshot shows a REST client interface with a request titled "Sara claims her reservation". The request is a POST to `http://localhost:8080/api/reservations/1/cl...`. The headers section shows "Content-Type" set to "application/json". The response is a 200 OK status with a body of "1 Reservation claimed, resource is now borrowed".

REQUEST HISTORY

- Today
 - POST http://localho...
 - POST POST http://l...
 - POST POST http://l...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
 - POST http://localho...
- June 20
 - GET http://localho...
 - GET http://localho...
 - GET http://localho...
- June 3
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...
 - GET http://192.16...

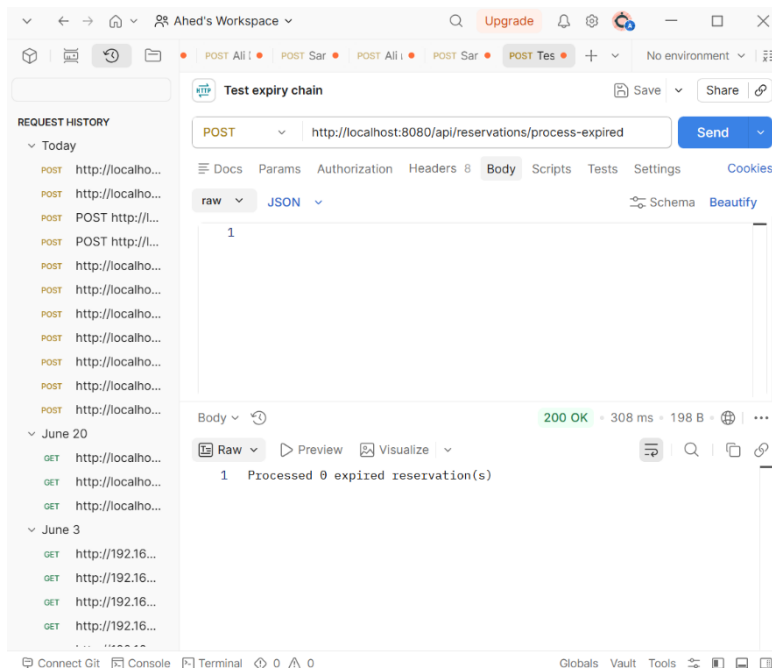
Headers

Key	Value	Bulk Edit	Presets
Content-Type	application/json		

Body

1 Reservation claimed, resource is now borrowed

7. Test expiry chain

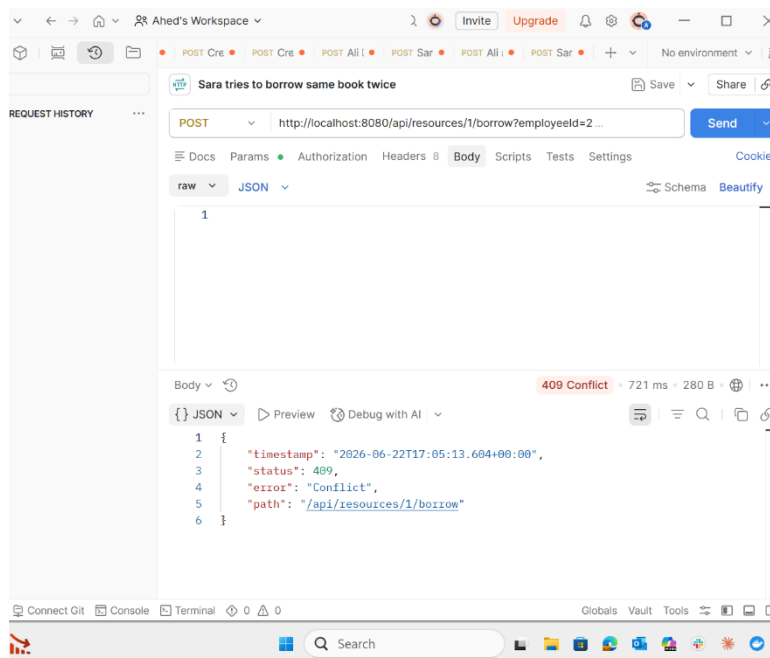


Test the business rule violations

These should all return errors, not success:

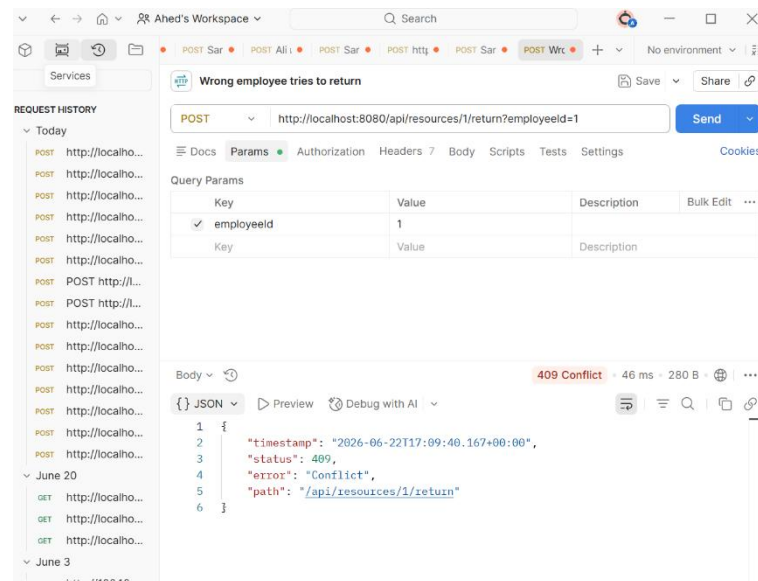
1— Sara tries to borrow same book twice → should get 409

POST <http://localhost:8080/api/resources/1/borrow?employeeid=1>



2— Wrong employee tries to return → should get 409 POST

<http://localhost:8080/api/resources/1/return?employeeId=2> (when Ali holds it)

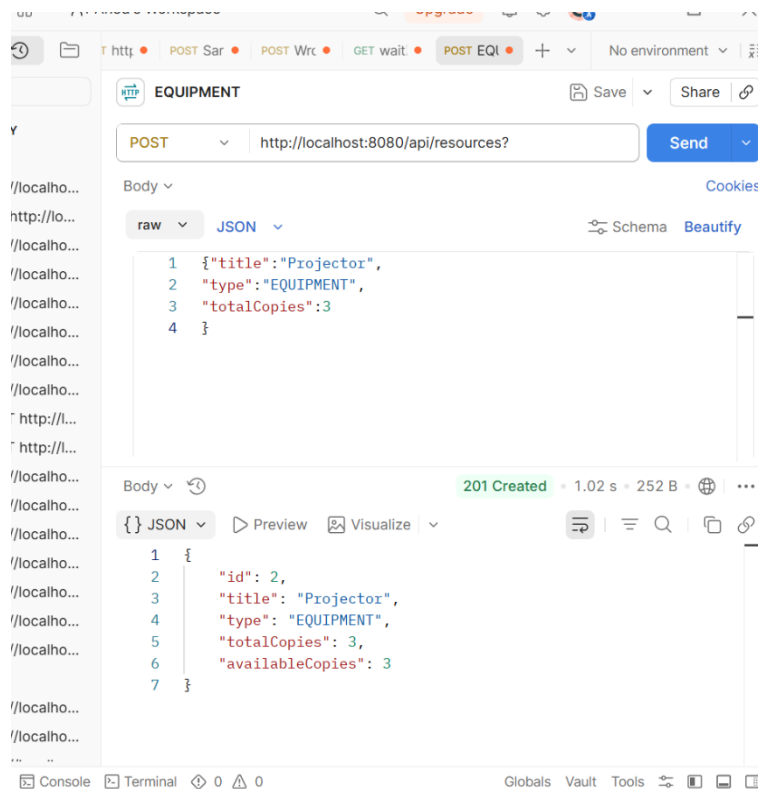


Adding other resource types:

For EQUIPMENT:

POST <http://localhost:8080/api/resources>

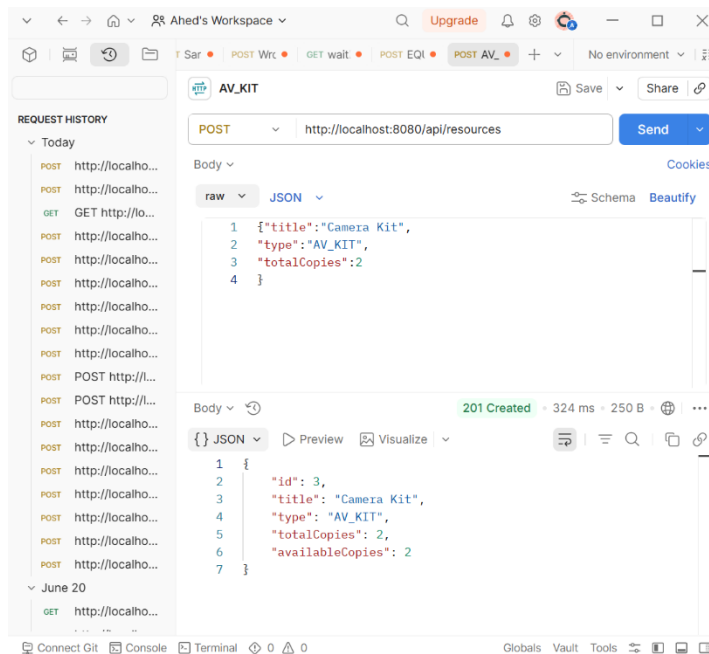
Body: `{"title":"Projector","type":"EQUIPMENT","totalCopies":3}`



For AV_KIT:

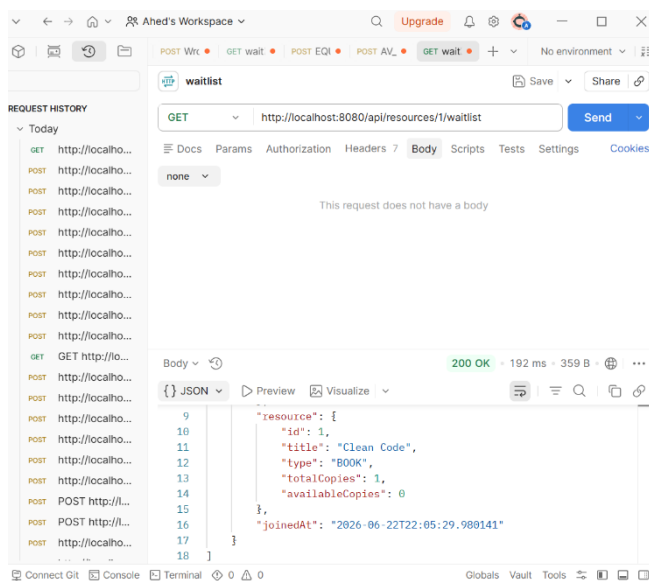
POST <http://localhost:8080/api/resources>

Body: {"title":"Camera Kit","type":"AV_KIT","totalCopies":2}



view the waitlist:

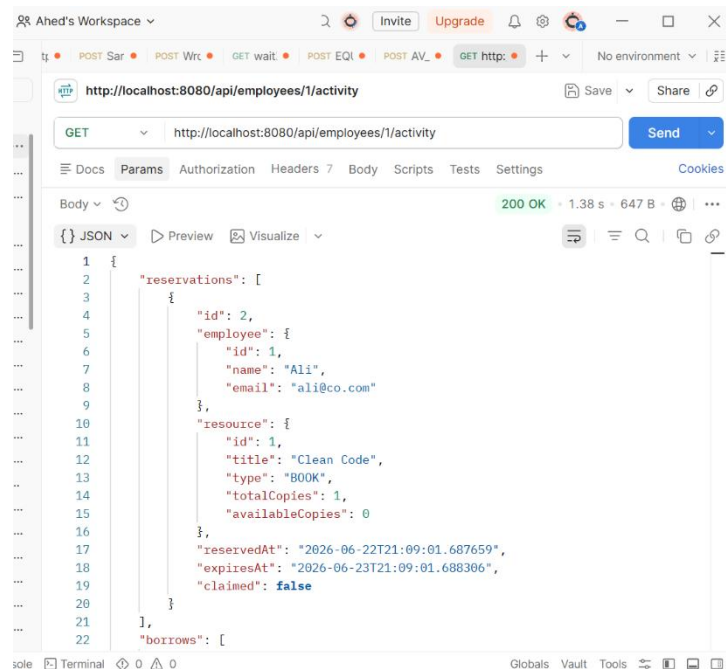
GET <http://localhost:8080/api/resources/1/waitlist>



GET — view employee current borrows and reservations:

GET <http://localhost:8080/api/employees/1/activity>

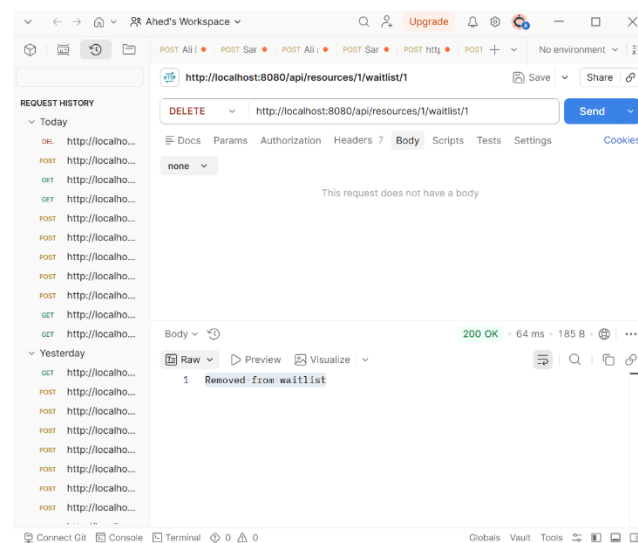
Returns what employee 1 currently holds and any pending reservations they have.



DELETE — leave waitlist voluntarily: "ali"

DELETE <http://localhost:8080/api/resources/1/waitlist/1>

This removes employee 3 from the waitlist for resource 1. Employee 2 stays in their position unaffected.



Ali:

GET <http://localhost:8080/api/employees/1/activity>

Sara:

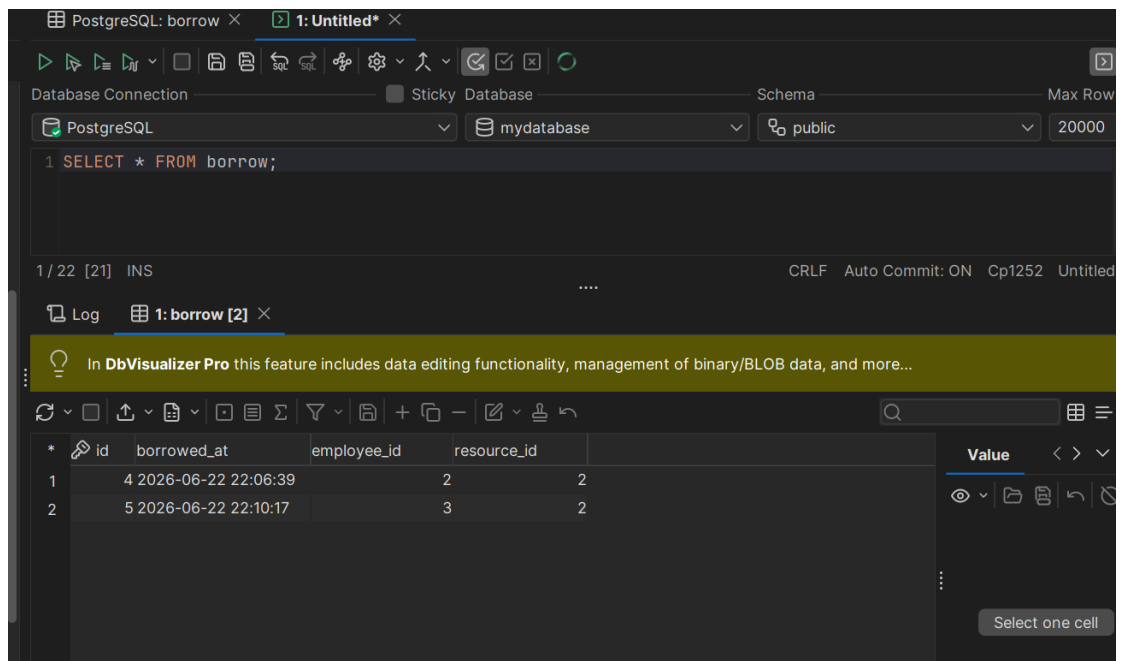
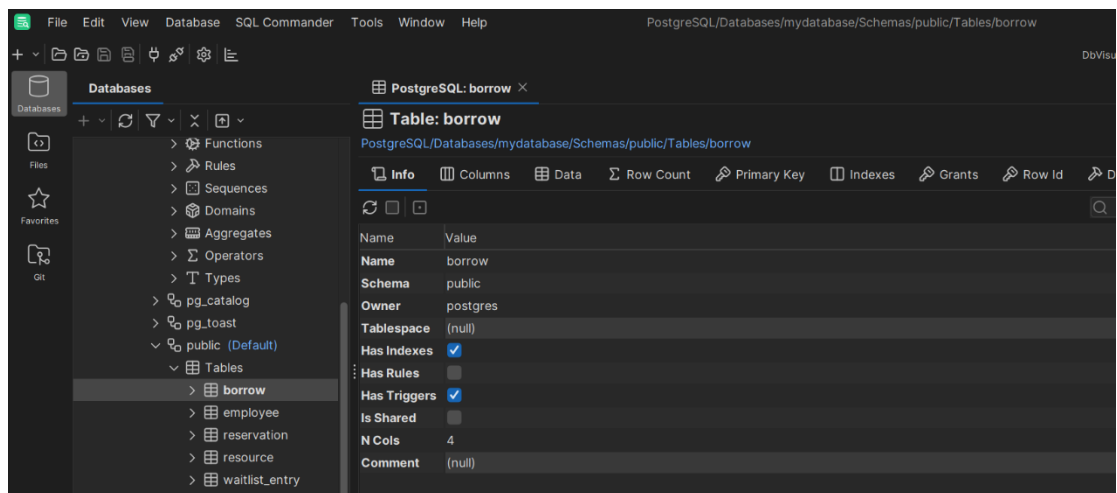
GET <http://localhost:8080/api/employees/2/activity>

Maria:

GET <http://localhost:8080/api/employees/3/activity>

to know which type of resources they have

DB part:



Database Connection: PostgreSQL, mydatabase, public, 20000

```
1 SELECT * FROM employee;
```

1 / 24 [23] INS CRLF Auto Commit: ON Cp1252 Untitled*

Log 1: employee [4] X

In DbVisualizer Pro this feature includes data editing functionality, management of binary/BLOB data, and more...

	id	email	name
1	1	1ali@co.com	Ali
2	2	2sara@co.com	Sara
3	3	3maria@co.com	Maria
4	4	4salam@co.com	Salam

Value < > X

Database Connection: PostgreSQL, mydatabase, public, 20000

```
1 SELECT * FROM resource;
```

1 / 24 [23] INS CRLF Auto Commit: ON Cp1252 Untitled*

Log 1: resource [3] X

In DbVisualizer Pro this feature includes data editing functionality, management of binary/BLOB data, and more...

	id	available_copies	title	total_copies	type
1	1	0	Clean Code	1	BOOK
2	3	2	Camera Kit	2	AV_KIT
3	2	0	Projector	3	EQUIPMENT

Value < > X

Database Connection: PostgreSQL, mydatabase, public, 20000

```
1 SELECT * FROM waitlist_entry;
```

1 / 30 [29] INS CRLF Auto Commit: ON Cp1252 Untitled*

Log 1: waitlist_entry [0] X

In DbVisualizer Pro this feature includes data editing functionality, management of binary/BLOB data, and more...

	id	joined_at	employee_id	resource_id
--	----	-----------	-------------	-------------

Result set is empty

Select one cell

PostgreSQL: reservation x 1:Untitled* x

Database Connection PostgreSQL Database mydatabase Schema public Max Rows 20000

1 SELECT * FROM reservation;

1 / 26 [25] INS CRLF Auto Commit: ON Cp1252 Untitled*

Log 1: reservation [3] x

In DbVisualizer Pro this feature includes data editing functionality, management of binary/BLOB data, and more...

	* id	claimed	expires_at	reserved_at	employee_id	resource_id	Value
1		1 true	2026-06-23 16:53:55	2026-06-22 16:53:55	2	1	
2		2 false	2026-06-23 21:09:01	2026-06-22 21:09:01	1	1	
3		3 false	2026-06-24 08:25:14	2026-06-23 08:25:14	4	2	

Select one cell